AMSS 2026 — Lab 2: Class Diagrams from Spec

Traian-Florin Șerbănuță

2026

Lab 2: Class Diagrams from AI

Three phases, 100 minutes — **you work solo this time.**

1. **Brief + spec** (~10 min) — read the 1-page spec; recall the read-order.
2. **Drive-and-critique drill** (~70 min) — drive AI to a class diagram, critique it, re-prompt twice.
3. **Share-out** (~20 min) — compare which defects the AI produced across the room.

Deliverable: a PlantUML class diagram + a critique log (1 page max), committed to the course lab repo.

Before You Start

- Continue.dev already working from Lab 1 — same endpoint, same config.
- You receive at lab start: your **student-id**, the lab-repo URL, and the **1-page spec** (also in `lab02/README.md` on clone).
- This is an **individual** lab. You play both roles — architect *and* critic — yourself.

The Domain: Library Kiosk

A neighbourhood library runs a self-service kiosk. This spec is **honest — no planted tricks.** The defects you will catch come from the AI, not the spec.

- The library owns a **catalogue of titles**. A *title* (book) has an ISBN, a title, and one or more authors.
- A popular title may have **several physical copies**. A *copy* has its own barcode and a condition. A copy belongs to the library and stays in the catalogue even when nobody has borrowed it.
- A **member** has a membership number, a name, and an email. Members come in two kinds: **standard** (up to 3 open loans) and **staff** (up to 10).
- To borrow, a member scans a **copy**. That opens a **loan**: one member, one copy, a start date, a due date (14 days on). A copy on loan cannot be borrowed by anyone else until returned.
- A member may have many loans over time, and several open at once (up to their limit). A loan always refers to **exactly one copy and one member**.
- On return, the member scans the copy: the loan closes with a

Your Job — the Loop

The same architect-critic loop from W4, run solo:

drive AI → read with the 4-step order → name the defects → re-prompt with domain rules → re-read.

- Bare first prompt, then **two required re-prompts** — iterate at least twice.
- Keep the **best** diagram, not the first draft.
- Log what you caught and why you re-prompted as you go — that log is the deliverable.

Round 1 — Bare Prompt + First Read

Send a deliberately bare prompt, pasting the spec:

> *"Generate a UML class diagram (as PlantUML) for this library system. [paste the 1-page spec]"*

Render it, then run the **4-step read-order** (next slide) against it.
Log every defect: its name, where it is, how bad it is.

The Read-Order (from W4)

A fixed order, every time:

1. Are the classes real **domain** concepts? (or invented infrastructure)
2. Are the **multiplicities** right? (read each one aloud)
3. Does each **association** name a real relationship? (a verb)
4. Are **whole-part** relationships captured? (aggregation / composition)

This is your critique-log rubric.

The Five Defects (from W4)

Name them with this vocabulary:

- **Wrong multiplicity** — `*--*` where it should be one (read it aloud).
- **Fake / decorative association** — a line with no nameable verb.
- **Missing aggregation** — a whole-part relationship drawn as a plain line.
- **Invented class / infrastructure** — `DatabaseManager`, `CacheController` — implementation, not domain.
- **God class** — one class that holds everything and does everything.

Re-prompt #1 — Name the Domain Rules

Fix the worst defects by stating the rules the AI got wrong. For this domain, that is almost always:

> *"A Book is a title; a Copy is a physical item — model both. A loan is exactly one copy to one member. The library holds many copies that outlive any loan (aggregation)."*

Regenerate, re-read with the 4-step order, log what changed. **This is iteration 1.**

Re-prompt #2 — Kill the Residue

Second pass targets what the first usually leaves:

> *"Drop any class that isn't a library-domain concept (no DatabaseManager, no god 'LibrarySystem'). Model member kinds as subclasses (standard / staff), not a type field."*

Regenerate, re-read, log. **This is iteration 2.** Then keep your best diagram.

Defect Card — Library Kiosk

If the draft looks complete, stress these. Each maps to a W4 defect:

- `Member "*" -- "*" Book` — **wrong multiplicity** (a loan is one copy to one member).
- Book and Copy as one class — collapses title vs physical item.
- `Library -- Copy` as a plain line — **missing aggregation** (the library *holds* copies).
- A line to `Member` with no verb — **fake association**.
- `LibrarySystem` with all the data + `doEverything()` — **god class**.
- `DatabaseManager`, `NotificationService` — **invented infrastructure**.
- Member kind as `type: String` — is-a flattened to an attribute.

Deliverable

On branch `lab02/<student-id>`, commit:

- `lab02/<student-id>/diagram.puml` — your best AI-driven PlantUML class diagram (must render).
- `lab02/<student-id>/critique-log.md` — 1 page max (next slide).
- `lab02/<student-id>/transcript.md` — optional but recommended: your raw prompt/output trail.

The Critique Log (1 page max)

Structured by the read-order. One short block per iteration (Round 1, re-prompt 1, re-prompt 2):

- **Defects found** — each: W4 name, where, severity.
- **Re-prompt move and why** — which domain rule you named and the reason. *(F3 — graded.)*
- **What changed** after regenerating.

Close with **one residual risk** — a defect the AI never got right.

How to Submit

```
git checkout -b lab02/<student-id>
mkdir -p lab02/<student-id>
# write diagram.puml and critique-log.md inside lab02/<stu
git add lab02/<student-id>
git commit -m "Lab 2: <student-id> library kiosk class diag
git push -u origin lab02/<student-id>
```

Push fails? Paste both files into the shared doc / email the instructor, then fix git after class.

Grading — Pass / Redo

Pass needs both:

1. `diagram.puml` (renders) and `critique-log.md` both committed by the deadline.
2. Your log names **at least two** distinct W4 defects correctly **and** gives a real reason for at least one re-prompt (not just what you typed).

A vacuous log ("the diagram was wrong, I fixed it") is a redo, not a fail. We grade your critique and reasoning — not the diagram's polish.

Share-out (20 min)

- A few students present the defect they caught that mattered most + the re-prompt move that fixed it.
- We tally, live, which defects the AI produced across the room.
- Same spec, same domain, ~100 of us → a real structural-defect map.

The defects you tally are exactly what you critique every week — and in the oral defense.

Why This Matters

A wrong class structure does not stay contained — it propagates to the sequence diagrams (W6), the tests, and the code.

Today you drilled **F1** (read & critique a diagram) and **F3** (say *why* you re-prompted). Next: **W5** widens to the other structural views; **Lab 3** flips you to red-team — hunting *planted* defects in artifacts we prepare.